FACEFORGE: TEAM INTEGRATION & CODE SPECIFICATION GUIDE

Final Year B.Tech IT (2026-27) · Group G2 | Project Module 4 Integration Contract

Notice to Teammates (Modules 1, 2, and 3):
Module 4 (Frontend User Interface, CSV Result Logging, Analytics Engine, and Adapter Shell) has been fully completed. To plug your code into the application without altering the UI or database, simply ensure your module matches the exact inputs and outputs specified below.

Module & Role	Responsible Teammate	Assigned File	Adapter Integration File
Module 1: Preprocessing	Teammate 1	module1_preprocess.py	adapter_preprocess.py
Module 2: Face Detection	Teammate 2	module2_detection.py	adapter_face.py
Module 3: Emotion Model	Teammate 3	module3_model.py + weights	adapter_emotion.py
Module 4: UI, Storage, Analytics	Module 4 Owner	app.py, storage.py, analytics.py	Fully Active (Complete)

1. TEAMMATE 1 — MODULE 1 (IMAGE PREPROCESSING)

Goal: Take an input BGR image frame and apply your preprocessing pipeline (e.g., resizing, histogram equalization, noise reduction).
Delivery File: module1_preprocess.py in the project root.

```
# module1_preprocess.py
import cv2
import numpy as np

def preprocess_image(img: np.ndarray) -> np.ndarray:
    '''
    Input: img (NumPy ndarray in BGR format from OpenCV)
    Output: preprocessed img (NumPy ndarray, BGR format)
    '''
    # Your processing logic here...
    return img
```

2. TEAMMATE 2 — MODULE 2 (FACE DETECTION & FRAMING)

Goal: Detect all faces in a BGR image frame and return bounding box coordinates.
Delivery File: module2_detection.py in the project root.

```
# module2_detection.py
import cv2
from typing import List, Tuple

def find_faces(img) -> List[Tuple[int, int, int, int]]:
    '''
    Input: BGR image ndarray
    Output: List of (x, y, w, h) bounding box coordinates
    Example: [(120, 80, 200, 200)]
    '''
    # Your Haar / YOLO / MTCNN detector logic here...
    return boxes
```

3. TEAMMATE 3 — MODULE 3 (EMOTION CLASSIFIER MODEL)

Goal: Classify a cropped face image into 7 emotion classes and produce probability scores.
Delivery Files: module3_model.py, model weights (e.g. .h5/.keras/.pt), and evaluation files in results/metrics_report.txt and results/confusion_matrix.png.
Required 7 Classes: angry, disgust, fear, happy, sad, surprise, neutral

```
# module3_model.py
from typing import Dict

def predict_emotion(face_crop) -> Dict[str, float]:
    '''
    Input: NumPy array of a cropped face
    Output: Dict of 7 probabilities that sum to ~1.0
    Example: {'angry': 0.05, 'disgust': 0.01, 'fear': 0.02, 'happy': 0.82,
              'sad': 0.03, 'surprise': 0.02, 'neutral': 0.05}
    '''
    # Your model.predict(face_crop) logic here...
    return scores
```

4. HOW TO TEST AND PLUG IN YOUR MODULE

Step 1: Put your .py script and model files into the FaceForge/ root folder.
Step 2: Open your corresponding adapter file (adapter_preprocess.py, adapter_face.py, or adapter_emotion.py).
Step 3: Import your function inside the adapter and return your results in the contract format.
Step 4: In config.yaml, switch your flag from false to true (e.g., use_module2: true).
Step 5: Run python app.py and check the UI at http://localhost:8000. If any import errors occur, the adapter will safely log a warning without crashing.

Prepared by: *Module 4 Owner (Integration & UI) · FaceForge Group G2 (2026-27)*